Jupyter Notebook Execution Report

Name: Amanda Quintino dos Santos
Project Title: BU OMDS
Date: June 28, 2026

Cell 1: ■ Code

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Lasso
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
```

Cell 2: ■ Code

```
predict_conversion_df = pd.read_csv("digital_marketing_campaign_dataset.csv")
google_ads_df = pd.read_csv("GoogleAds_DataAnalytics_Sales_Uncleaned.csv")
marketing_products_df = pd.read_csv("marketing_and_product_performance.csv")
```

Cell 3: ■ Markdown

Feature Scaling predict_conversion_df

Cell 4: ■ Code

```
# Identify numerical columns excluding ID and target variable
num_cols = predict_conversion_df.select_dtypes(include=['int64',
'float64']).columns
cols_to_exclude = ['CustomerID', 'Conversion']
cols_to_scale = [col for col in num_cols if col not in cols_to_exclude]
```

```
# StandardScaler
scaler = StandardScaler()
```

```
# Fit and transform the numerical features
predict_conversion_df_scaled = predict_conversion_df.copy()
predict_conversion_df_scaled[cols_to_scale] =
scaler.fit_transform(predict_conversion_df[cols_to_scale])
```

```
# Update the dataframe
predict_conversion_df = predict_conversion_df_scaled
```

```
# Display the scaled dataframe
predict_conversion_df.head()
```

Output:

	CustomerID	Age	Gender	...	AdvertisingPlatform	AdvertisingTool	Conversion
0	8000	0.830400	Female	...	IsConfid	ToolConfid	1
1	8001	1.702775	Male	...	IsConfid	ToolConfid	1
2	8002	0.159343	Female	...	IsConfid	ToolConfid	1
3	8003	-0.780138	Female	...	IsConfid	ToolConfid	1
4	8004	1.098823	Female	...	IsConfid	ToolConfid	1

[5 rows x 20 columns]

Cell 5: ■ Markdown

Feature Scaling for google_ads_df

Cell 6: ■ Code

```
# Clean 'Cost' and 'Sale_Amount' by converting to float
for col in ['Cost', 'Sale_Amount']:
if google_ads_df[col].dtype == 'object':
google_ads_df[col] = google_ads_df[col].str.replace('$', '',
regex=False).astype(float)
```

```
# Identify numerical columns for scaling
google_ads_num_cols = google_ads_df.select_dtypes(include=['float64',
'int64']).columns
google_ads_cols_to_exclude = [] # Exclude any identifiers or targets if needed
google_ads_cols_to_scale = [col for col in google_ads_num_cols if col not in
google_ads_cols_to_exclude]
```

```
# Fit and transform the numerical features
google_ads_df_scaled = google_ads_df.copy()
google_ads_df_scaled[google_ads_cols_to_scale] =
scaler.fit_transform(google_ads_df[google_ads_cols_to_scale])
```

```
# Update the dataframe
google_ads_df = google_ads_df_scaled
```

```
# Display the scaled dataframe
google_ads_df.head()
```

Output:

```
   Ad_ID          Campaign_Name  ...   Device                 Keyword
0  A1000    DataAnalyticsCourse  ...  desktop    learn data analytics
1  A1001    DataAnalyticsCourse  ...   mobile   data analytics course
2  A1002    Data Anlytics Corse  ...  Desktop   data analitics online
3  A1003  Data Analytcis Course  ...   tablet  data anaytics training
4  A1004   Data Analytics Corse  ...  desktop    online data analytic

[5 rows x 13 columns]

[STDERR]
&lt;string&gt;:4: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but whe
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/ind

&lt;string&gt;:4: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in certain cases, but whe
A typical example is when you are setting values in a column of a DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/ind
```

Cell 7: ■ Markdown

Feature Scaling for marketing_products_df

Cell 8: ■ Code

```
# Identify numerical columns for scaling
marketing_num_cols = marketing_products_df.select_dtypes(include=['float64', 'int64']).columns
marketing_cols_to_exclude = [] # No numeric ID or target features to exclude
marketing_cols_to_scale = [col for col in marketing_num_cols if col not in marketing_cols_to_exclude]
```

```
# Fit and transform the numerical features
marketing_products_df_scaled = marketing_products_df.copy()
marketing_products_df_scaled[marketing_cols_to_scale] = scaler.fit_transform(marketing_products_df[marketing_cols_to_scale])
```

```
# Update the dataframe
marketing_products_df = marketing_products_df_scaled
```

```
# Display the scaled dataframe
marketing_products_df.head()
```

Output:

```
  Campaign_ID   Product_ID  ...  Customer_Satisfaction_Post_Refund  Common_Keywords
0  CMP_RLSDVN  PROD_HBJFA3  ...                           1.346666       Affordable
1  CMP_JHHUE9  PROD_OE8YNJ  ...                          -0.449967       Innovative
2  CMP_6SBOWN  PROD_4V8A08  ...                           1.346666       Affordable
3  CMP_Q31QCU  PROD_A1Q6ZB  ...                          -1.348283          Durable
4  CMP_AY0UTJ  PROD_F57N66  ...                          -0.449967       Affordable

[5 rows x 17 columns]
```

Cell 9: ■ Markdown

Logistic Regression of Datasets

Cell 10: ■ Code

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score
import warnings
warnings.filterwarnings('ignore')
```

```
# 1. Logistic Regression for predict_conversion_df
print("=== predict_conversion_df ===")
# Prepare features (convert categorical string columns to dummy variables)
X1 = pd.get_dummies(predict_conversion_df.drop(columns=['CustomerID',
'Conversion']), drop_first=True)
y1 = predict_conversion_df['Conversion']
```

```
X1_train, X1_test, y1_train, y1_test = train_test_split(X1, y1, test_size=0.2,
random_state=42)
```

```
lr1 = LogisticRegression(max_iter=1000)
lr1.fit(X1_train, y1_train)
y1_pred = lr1.predict(X1_test)
print(f"Accuracy: {accuracy_score(y1_test, y1_pred):.4f}")
print("Classification Report:\n", classification_report(y1_test, y1_pred))
```

```
# 2. Logistic Regression for google_ads_df
print("\n=== google_ads_df ===")
# Target: Did the ad lead to any conversions?
google_ads_clean = google_ads_df.dropna() # Drop rows with NaNs (e.g. Conversion
Rate) to simplify
X2 = pd.get_dummies(google_ads_clean.drop(columns=['Ad_ID', 'Campaign_Name',
'Ad_Date', 'Keyword', 'Location', 'Device', 'Conversions', 'Conversion Rate']),
drop_first=True)
y2 = (google_ads_clean['Conversions'] &gt; 0).astype(int)
```

```
if len(y2.unique()) &gt; 1:
X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2, test_size=0.2,
random_state=42)
lr2 = LogisticRegression(max_iter=1000)
lr2.fit(X2_train, y2_train)
y2_pred = lr2.predict(X2_test)
print(f"Accuracy: {accuracy_score(y2_test, y2_pred):.4f}")
else:
```

```
print("Not enough class variation for Logistic Regression.")
```

```
# 3. Logistic Regression for marketing_products_df
print("\n=== marketing_products_df ===")
# Target: Did the campaign lead to more than average conversions? Or &gt; 0?
# Assuming we predict if there are conversions &gt; 0 (or &gt; median since all might have &gt; 0)
median_conv = marketing_products_df['Conversions'].median()
y3 = (marketing_products_df['Conversions'] &gt; median_conv).astype(int)
X3 = pd.get_dummies(marketing_products_df.drop(columns=['Campaign_ID', 'Product_ID', 'Customer_ID', 'Flash_Sale_ID', 'Bundle_ID', 'Conversions', 'Common_Keywords', 'Subscription_Tier']), drop_first=True)
```

```
X3_train, X3_test, y3_train, y3_test = train_test_split(X3, y3, test_size=0.2, random_state=42)
```

```
lr3 = LogisticRegression(max_iter=1000)
lr3.fit(X3_train, y3_train)
y3_pred = lr3.predict(X3_test)
print(f"Accuracy: {accuracy_score(y3_test, y3_pred):.4f}")
print("Classification Report:\n", classification_report(y3_test, y3_pred))
```

Output:

```
=== predict_conversion_df ===
Accuracy: 0.8869
Classification Report:
              precision    recall  f1-score   support

           0       0.62      0.18      0.27       194
           1       0.90      0.99      0.94      1406

    accuracy                           0.89      1600
   macro avg       0.76      0.58      0.61      1600
weighted avg       0.86      0.89      0.86      1600

=== google_ads_df ===
Accuracy: 0.5088
=== marketing_products_df ===
Accuracy: 0.5140
Classification Report:
```

	precision	recall	f1-score	support
0	0.54	0.44	0.48	1033
1	0.50	0.59	0.54	967
accuracy			0.51	2000
macro avg	0.52	0.52	0.51	2000
weighted avg	0.52	0.51	0.51	2000